

실전반 3회 · Neo4j로 설비 관계 연결

- [이번 회차에서 할 일](#)
- [1. 시작 전 준비 \(복습·환경 확인\)](#)
- [2. 핵심 개념](#)
- [3. 활동 1 — 세 가지 Golden Question과 Asset·Sensor·SOP 관계 정하기 · 50분](#)
- [4. 활동 2 — Resource·Measure 대응을 읽고 제공 MERGE 문으로 적재 · 50분](#)
- [5. 활동 3 — 파라미터 Cypher로 설비에서 SOP까지 탐색 · 50분](#)
- [6. 활동 4 — asset_id로 그래프와 시계열 결과를 결합 · 50분](#)
- [7. 실습 과제](#)
- [8. 완료 확인 체크리스트](#)
- [9. 회고 \(10분\)](#)
- [강사 노트](#)
- [용어](#)

날짜·시간	2026-11-21 (토) 14:00~18:00	모듈	M1 · 원 일정 6번
블록	구현 B4 · 확장 B3	산출물	B3+B4 설비 맥락 조회
완료 확인	다른 설비의 SOP가 섞이지 않는 관계 질의		

회차별 블록 로드맵 — 하나의 시스템에 매 회차 기능을 더한다

구현 새로 만든다 · **확장** 조건·연결을 바꾼다 · **플랫폼** 교육용 플랫폼에 연결한다

실전반 4학년 · 9회 × 4시간 (합수·질의·정책 수정과 검증)

	B1	B2	B3	B4	B5	B6	B7	B8	B9
1회 10/31	구현	구현	구현						
2회 11/07		확장	확장						
3회 11/21			확장	구현					
4회 11/28		확장	확장		구현			구현	
5회 01/25				확장		구현	구현		
6회 01/26			플랫폼	확장+P			확장		
7회 01/27						확장+P	확장+P	구현+P	
8회 01/28	확장				확장			확장	
9회 01/29									확장+P

통합반 2·3학년 · 10회 × 4시간 (제공 코드의 매핑·설정·조건 완성)

	B1	B2	B3	B4	B5	B6	B7	B8	B9
1회 11/14	구현	구현							
2회 01/25		확장	구현						
3회 01/26				구현					
4회 01/27		확장	확장		구현			구현	
5회 01/28				확장		구현			
6회 01/29							구현	구현	
7회 02/01			플랫폼	확장+P					
8회 02/02						확장+P	확장+P	확장+P	
9회 02/03									확장+P
10회 02/04	확장				확장			확장	확장

이번 회차가 다루는 블록

이번 회차에서 할 일

1·2회차에서 PostgreSQL 에 쌓은 관측은 "HYD-02 의 TS1 이 53°C" 까지만 말한다. 그 설비에 어떤 센서가 달려 있는지, 냉각 이상이 나면 어떤 SOP 를 따라야 하고 어떤 조치가 허용되는지는 수치 테이블에 없다. 오늘은 Ontology 계층의 B4 를 구현해 설비(Asset)·센서(Sensor)·SOP·조치(Action)의 관계를 Neo4j 에 두고, 파라미터 Cypher 질의로 설비에서 SOP 까지 탐색한다. 그리고 그래프 결과와 B3 의 최근 구간을 `asset_id` 로 결합한다. 핵심 요구는 하나다 — **HYD-02(공랭식)에 HYD-01(수랭식)의 SOP 가 절대 섞이지 않아야 한다.** 섞이면 5회차 에이전트가 2호기에 1호기의 부하 감소 절차를 근거로 조치를 제안한다.

학습 목표

- 세 가지 Golden Question 을 답하는 데 필요한 노드와 관계를 표로 정한다.
- 설비·센서가 Ontologic 의 Resource·Measure 에 어떻게 대응하는지, MERGE 적재가 왜 여러 번 실행해도 결과가 같은지 설명한다.
- 설비 센서 / 적용 SOP·허용 조치 / 설비 맥락의 세 파라미터 Cypher 를 고치고, 설비·버전 필터가 빠졌을 때 섞이는 SOP 를 실제 출력으로 보인다.
- 그래프의 센서 목록과 PostgreSQL 최근 60초 요약을 `asset_id` 와 센서 코드로 결합한다.

1. 시작 전 준비 (복습·환경 확인)

```
cd lecture/system
PYTHONPATH=. .venv/bin/python -m pytest ../labs/practical/S01
../labs/practical/S02 -q
```

두 과제를 마쳤다면 `11 passed` 가 나온다. Neo4j 가 강사 적재 상태인지 읽기 전용으로 확인한다.

```
PYTHONPATH=. .venv/bin/python -c "from hydops.b4_ontology import graph;
print(graph.graph_stats())"
```

2026-09-14 실행 결과:

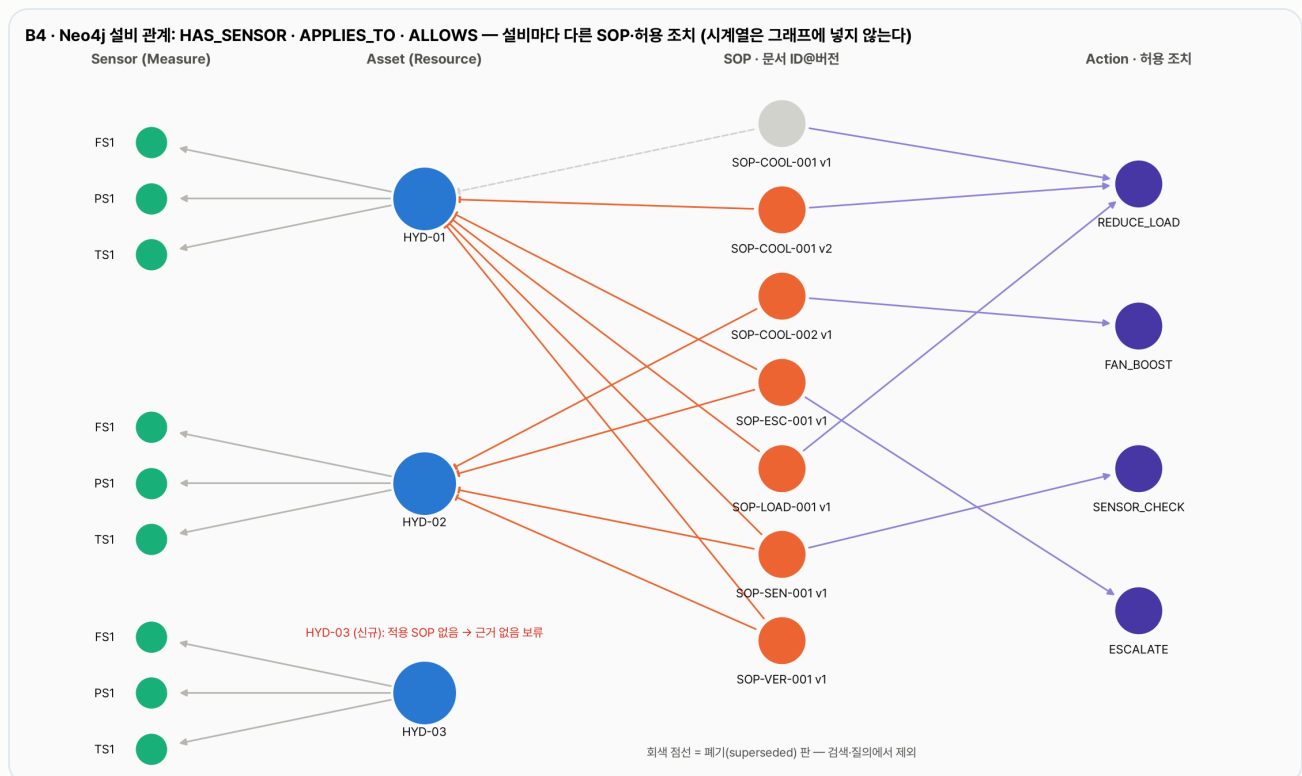
```
{'nodes': {'Asset': 3, 'Component': 3, 'Sensor': 9, 'SOP': 7, 'SOPSection': 42,
'Action': 4, 'Event': 6, 'Driver': 2, 'Process': 2, 'KPI': 2}, 'relationships':
{'HAS_COMPONENT': 3, 'HAS_SENSOR': 9, 'RUNS_ON': 6, 'MEASURED_FOR': 6,
'APPLIES_TO': 10, 'ALLOWS': 6, 'HAS_SECTION': 42, 'SUPERSEDES': 1, 'ON_ASSET': 6,
'CITES': 15, 'AFFECTS': 6}}
```

Event · ON_ASSET · CITES 수는 관제 서버가 만든 사건 참조라 실행 시점마다 다르다. 나머지는 적재 데이터로 정해진다. graph_stats 는 라벨을 하나씩 지정해 센다(Asset , Component , Sensor ...). 노드 하나가 Asset:Resource 처럼 라벨을 두 개 가질 수 있으므로, "첫 라벨"로 세면 냉각 구성품(Component:Resource)이 Resource 로 잘못 보일 수 있다.

Neo4j Browser 는 <http://localhost:57474> (neo4j / hydops-lecture)이다.

2. 핵심 개념

2.1 그래프에 무엇을 두고 무엇을 두지 않는가



Neo4j 설비 관계

설비 3대, 설비마다 센서 3개, SOP 7개(버전 포함), 조치 4개다. 주황 선이 APPLIES_TO , 보라 선이 ALLOWS 다. HYD-01 에는 SOP-COOL-001 v2·SOP-LOAD-001 이, HYD-02 에는 SOP-COOL-002 가 따로 걸린다. 회색 점선은 폐기된 SOP-COOL-001 v1 이다. HYD-03(신규)에는 SOP 가 하나도 없다.

```
# lecture/system/hydops/b4_ontology/graph.py (발췌)
```

```
"""B4 · Neo4j 설비 관계와 업무 개념.
```

```
시계열 전체를 그래프 노드로 만들지 않는다. 설비·센서·SOP·조치와 사건 참조만 둔다.
```

수치 상제는 PostgreSQL 에서 asset_id 로 조회한다.

Ontologic 대응: Resource=유압설비·냉각기, Measure=온도·유량·압력, Driver=부하·냉각 성능, Process=운전·점검, KPI=이상 지속 시간·불필요 조치 건수. Asset 은 Resource 의 수업용 구체 클래스다.

""""

주의 — 관계를 그렸다고 실측 인과가 확인된 것이 아니다. 시드의 AFFECTS 관계에는 근거 종류가 속성으로 붙어 있다: 냉각 성능→온도는 basis='document' , 부하→온도는 basis='simulator_assumption' , 둘 다 verified_causal=false 다.

2.2 세 가지 Golden Question

hydops/golden.py 는 그래프(B4)와 시계열(B3)을 asset_id · event_id 로 결합해 세 질문에 답한다.

```
# lecture/system/hydops/golden.py (발췌)
"""세 가지 Golden Question 을 그래프(B4)와 시계열(B3)을 asset_id·event_id 로 결합해 답한다.

1. HYD-01의 최근 온도는 유효한 센서 관측이며 지속 이상인가?
2. 이 설비에 적용되는 최신 냉각 이상 SOP와 허용 조치는 무엇인가?
3. 어떤 근거로 누가 승인했으며 조치 후 새 관측은 회복했는가?
"""
```

오늘은 1·2번의 그래프 부분과 1번의 결합을 만든다. 3번은 승인·조치 기록이 생기는 7·8회차에 완성된다.

3. 활동 1 — 세 가지 Golden Question과 Asset·Sensor·SOP 관계 정하기 · 50분

목표

질문마다 필요한 노드·관계·필터·저장소를 정해 팀의 관계 표를 만든다.

따라 하기

1. 팀별로 질문 하나씩 맡아 "이 답을 내려면 무엇을 따라가야 하는가"를 적는다. 완성 시스템 기준 답은 다음 표와 같다.

질문	그래프 경로	필터	PostgreSQL
Q1 최근 온도가 유효·지속 이상인가	(Asset)-[:HAS_SENSOR]->(Sensor {quantity:'temperature'}) → 센서 코드	asset_id	observation 최근 60초, 품질 플래그
Q2 적용 최신 SOP·허용 조치	(SOP)-[:APPLIES_TO]->(Asset) , (SOP)-[:ALLWS]->(Action)	asset_id , status='active' , event_type	—
Q3 근거·승인·회복	(Event)-[:ON_ASSET]->(Asset) , (Event)-[:CITES]->(SOPSection)	event_id	approval , action_log , verification

2. 설비별 SOP 적용 관계를 직접 조회해 표의 Q2 경로를 확인한다(읽기 전용).

```
docker compose exec -T neo4j cypher-shell -u neo4j -p hydops-lecture \
  "MATCH (s:SOP)-[:APPLIES_TO]->(a:Asset) RETURN a.asset_id AS asset,
  collect(s.sop_key + '(' + s.status + ')') AS sops ORDER BY asset;"
```

```
asset, sops
"HYD-01", ["SOP-VER-001@v1(active)", "SOP-SEN-001@v1(active)", "SOP-LOAD-001@v1(active)", "SOP-ESC-001@v1(active)", "SOP-COOL-001@v2(active)", "SOP-COOL-001@v1(superseded)"]
"HYD-02", ["SOP-VER-001@v1(active)", "SOP-SEN-001@v1(active)", "SOP-ESC-001@v1(active)", "SOP-COOL-002@v1(active)"]
```

HYD-03 은 결과에 행이 없다. APPLIES_TO 가 하나도 없기 때문이다.

3. 문서 쪽 근거를 확인한다. SOP-COOL-002 는 적용 범위를 문서에 명시한다.

```

<!-- lecture/system/hydops/b6_sop/docs/SOP-C00L-002_v1.md (발책) -->
applies_to: [HYD-02]
event_type: COOLING_ANOMALY
allows: [FAN_BOOST]
...
## 1. 적용 설비
HYD-02 공랭식 유압설비에만 적용한다. HYD-01에는 적용하지 않는다.
## 4. 허용 조치
허용 조치는 냉각 팬 증속(FAN_BOOST)으로 팬 속도 배수를 1.3으로 올린다. 허용 값 범위는 1.0 이상
1.5 이하이다. 부하 감소는 이 절차의 허용 조치가 아니다.

```

확인 포인트

- Q2 는 설비 필터와 버전(status) 필터가 둘 다 있어야 한다. HYD-01 에는 active 와 superseded 가 함께 걸려 있다.
- 시계열 값은 그래프에 없다. Q1 은 그래프에서 센서를 찾고 값은 PostgreSQL 에서 읽는다.

흔한 실수

- SOP 를 설비 속성(`a.sops = [...]`)으로 저장한다. 그러면 SOP 버전·허용 조치·절(section)과의 관계를 따라갈 수 없다.
- 관측 1초마다 (`:Observation`) 노드를 만든다. 사이클 하나가 180행이므로 그래프가 수치 저장소가 되어 버린다(D07 저장 분리 원칙).

4. 활동 2 — Resource·Measure 대응을 읽고 제공 MERGE 문으로 적재 · 50분

목표

제공 `seed_graph` 의 MERGE 문을 읽고, 라벨 두 개(`Asset:Resource` , `Sensor:Measure`)와 전역 센서 ID 규칙을 확인한다.

따라 하기

1. 설비·센서 적재 문장을 읽는다.

```
# lecture/system/hydops/b4_ontology/graph.py (발췌)
for a in ASSETS:
    run(
        """MERGE (a:Asset {asset_id:$asset_id})
           SET a:Resource, a.name=$name, a.cooling=$cooling, a.line=$line,
           a.educational=$educational
           MERGE (c:Component:Resource {component_id:$asset_id + '.COOLER'})
           SET c.name=$cooling
           MERGE (a)-[:HAS_COMPONENT]->(c)""",
        **a,
    )
    for s in SENSORS:
        run(
            """MATCH (a:Asset {asset_id:$asset_id})
               MERGE (s:Sensor {sensor_id:$asset_id + '.' + $code})
               SET s:Measure, s.code=$code, s.quantity=$quantity,
               s.unit=$unit, s.hz=$hz, s.name=$name
               MERGE (a)-[:HAS_SENSOR]->(s)""",
            asset_id=a["asset_id"],
            **s,
        )
```

수업 클래스	Ontologic 관 점	키	예
Asset	Resource	asset_id	HYD-01
Component	Resource	component_id	HYD-01.COOLER
Sensor	Measure	sensor_id = asset_id + '.' + code	HYD-01.TS1
Driver	Driver	driver_id	LOAD , COOLING_PERF
SOP	문서	sop_key = doc_id@v버전	SOP-C00L-001@v2

2. 적재된 노드의 라벨과 속성을 확인한다.

```
graph.run('MATCH (a:Asset {asset_id:$a}) RETURN labels(a) AS labels,
          a.asset_id AS id', a='HYD-01')
graph.run('MATCH (s:Sensor {sensor_id:$s}) RETURN labels(s) AS labels, s
          {.*} AS props', s='HYD-01.TS1')
```

```
[{'labels': ['Asset', 'Resource'], 'id': 'HYD-01'}]
[{'labels': ['Sensor', 'Measure'], 'props': {'sensor_id': 'HYD-01.TS1',
```

```
'unit': '°C', 'hz': 1, 'quantity': 'temperature', 'code': 'TS1', 'name': '탱크 온도'}}}]
```

센서 노드의 `sensor_id` 는 설비가 붙은 전역 ID(`HYD-01.TS1`)이고, 관측 테이블의 `sensor_id` 는 코드(`TS1`)다. 활동 4 의 결합 키가 여기서 갈린다.

3. MERGE 가 여러 번 실행해도 결과가 같은 이유를 확인한다. `CONSTRAINTS` 가 키마다 유일 제약을 건다.

```
# lecture/system/hydops/b4_ontology/graph.py (발췌)
CONSTRAINTS = [
    "CREATE CONSTRAINT asset_id IF NOT EXISTS FOR (n:Asset) REQUIRE
      n.asset_id IS UNIQUE",
    "CREATE CONSTRAINT sensor_id IF NOT EXISTS FOR (n:Sensor) REQUIRE
      n.sensor_id IS UNIQUE",
    "CREATE CONSTRAINT sop_key IF NOT EXISTS FOR (n:SOP) REQUIRE n.sop_key
      IS UNIQUE",
    ...
]
```

`seed_graph()` 는 마지막에 `graph_stats()` 를 돌려주므로, 강사 적재가 끝난 그래프에 같은 값으로 다시 적재해도 §1 에서 본 적재 데이터 수치(Asset 3 · Component 3 · Sensor 9 · SOP 7 · APPLIES_TO 10 · ALLOWS 6 · AFFECTS 6 · SUPERSEDES 1)가 그대로여야 한다. 수업 그래프는 여러 팀이 함께 쓰므로 **적재는 강사가 한 번 하고, 학생은 읽기 질의만 한다.**

4. 버전 관계와 관계의 근거 속성을 확인한다.

```
graph.run('MATCH (new:SOP)-[:SUPERSEDES]->(old:SOP) RETURN new.sop_key AS
  new, new.status AS ns, old.sop_key AS old, old.status AS os')
graph.run('MATCH (d:Driver)-[:AFFECTS]->(s:Sensor) RETURN d.driver_id AS
  driver, s.sensor_id AS sensor, r.basis AS basis, r.source AS source,
  r.verified_causal AS verified ORDER BY driver, sensor')
```

```
[{'new': 'SOP-COOL-001@v2', 'ns': 'active', 'old': 'SOP-COOL-001@v1', 'os':
  'superseded'}]
[{'driver': 'COOLING_PERF', 'sensor': 'HYD-01.TS1', 'basis': 'document',
  'source': 'SOP-COOL-001@v2 §3', 'verified': False}, {'driver':
  'COOLING_PERF', 'sensor': 'HYD-02.TS1', 'basis': 'document', 'source': 'SOP-
  COOL-002@v1 §2', 'verified': False}, {'driver': 'COOLING_PERF', 'sensor':
  'HYD-03.TS1', 'basis': 'simulator_assumption', 'source': 'simulator.py T_ss
  식 (적용 SOP 없음)', 'verified': False}, {'driver': 'LOAD', 'sensor': 'HYD-
  01.TS1', 'basis': 'simulator_assumption', 'source': 'simulator.py T_ss 식',
  'verified': False}, {'driver': 'LOAD', 'sensor': 'HYD-02.TS1', 'basis':
  'simulator_assumption', 'source': 'simulator.py T_ss 식', 'verified': False},
```

```
{'driver': 'LOAD', 'sensor': 'HYD-03.TS1', 'basis': 'simulator_assumption',  
'source': 'simulator.py T_ss 식', 'verified': False}]
```

냉각 성능 → 온도 관계의 근거는 설비마다 다르다. 적재 문장이 설비별 근거 표를 두고, 그 설비의 센서만 잡는다.

```
# lecture/system/hydops/b4_ontology/graph.py (발췌)  
basis_by_asset = {  
    "HYD-01": ("document", "SOP-COOL-001@v2 §3"),  
    "HYD-02": ("document", "SOP-COOL-002@v1 §2"),  
}  
run("MATCH (:Driver)-[r:AFFECTS]->(:Sensor) DELETE r")  
for a in ASSETS:  
    basis, source = basis_by_asset.get(a["asset_id"],  
        ("simulator_assumption", "simulator.py T_ss 식 (적용 SOP 없음)"))  
    run(  
        """MATCH (cool:Driver {driver_id:'COOLING_PERF'}), (load:Driver  
        {driver_id:'LOAD'})  
        MATCH (:Asset {asset_id:$asset_id})-[:HAS_SENSOR]->(s:Sensor  
        {code:'TS1'})  
        MERGE (cool)-[r1:AFFECTS]->(s) SET r1.basis=$basis,  
        r1.source=$source, r1.verified_causal=false  
        MERGE (load)-[r2:AFFECTS]->(s) SET  
        r2.basis='simulator_assumption', r2.source='simulator.py T_ss 식',  
        r2.verified_causal=false""",  
        asset_id=a["asset_id"], basis=basis, source=source,  
    )
```

HYD-02의 근거는 HYD-02에 적용되는 SOP-COOL-002, HYD-03은 적용 SOP가 없으므로 문서 근거 대신 시뮬레이터 가정으로 적혀 있다. 토론 거리: **근거 문서는 그 설비에 적용되는 SOP 여야 한다.** 이 시스템의 초기 시드는 (s:Sensor {code:'TS1'})로 세 설비의 TS1을 한꺼번에 잡아, HYD-02·HYD-03의 관계에도 HYD-01 전용 문서 SOP-COOL-001@v2 §3이 출처로 잘못 연결된 적이 있다. SOP의 APPLIES_TO 뿐 아니라 관계의 출처 속성에서도 설비 범위가 섞일 수 있다.

확인 포인트

- Asset은 Resource 라벨을, Sensor는 Measure 라벨을 함께 가진다.
- SOP는 (doc_id, version)마다 노드 하나이고, v2가 v1을 SUPERSEDES한다.
- AFFECTS의 문서 근거(basis='document')는 그 설비에 APPLIES_TO로 걸린 SOP만 가리킨다. 적용 SOP가 없는 HYD-03은 simulator_assumption이다.

흔한 실수

- **MERGE** 를 **CREATE** 로 바꿔 여러 번 실행한다. 유일 제약이 있으면 두 번째 실행부터 제약 위반 오류로 멈추고, 제약이 없는 라벨(예: **Driver**)은 같은 노드가 중복된다.
- 센서 노드 키를 코드(**TS1**)로만 둔다. 세 설비의 TS1 이 한 노드로 합쳐져 **HAS_SENSOR** 가 설비 셋에서 한 센서로 모인다.

5. 활동 3 — 파라미터 Cypher로 설비에서 SOP까지 탐색 · 50분

목표

`labs/practical/S03/context_queries.py` 의 세 질의를 고쳐, 설비·사건 유형 파라미터로 그 설비의 최신 SOP 와 허용 조치만 돌려받는다.

따라 하기

1. 완성 시스템의 적용 SOP 질의를 읽는다.

```
// lecture/system/hydops/b4_ontology/graph.py Q_APPLICABLE_SOP (발책)
MATCH (sop:SOP)-[:APPLIES_TO]->(a:Asset {asset_id:$asset_id})
WHERE sop.status = 'active' AND sop.event_type = $event_type
OPTIONAL MATCH (sop)-[:ALLWS]->(act:Action)
RETURN sop.doc_id AS doc_id, sop.version AS version, sop.title AS title,
       toString(sop.effective_date) AS effective_date, sop.approver_role AS
approver_role,
       collect(DISTINCT act {.action_id, .name, .min_value, .max_value,
.default_value, .requires_approval}) AS allowed_actions
ORDER BY doc_id
```

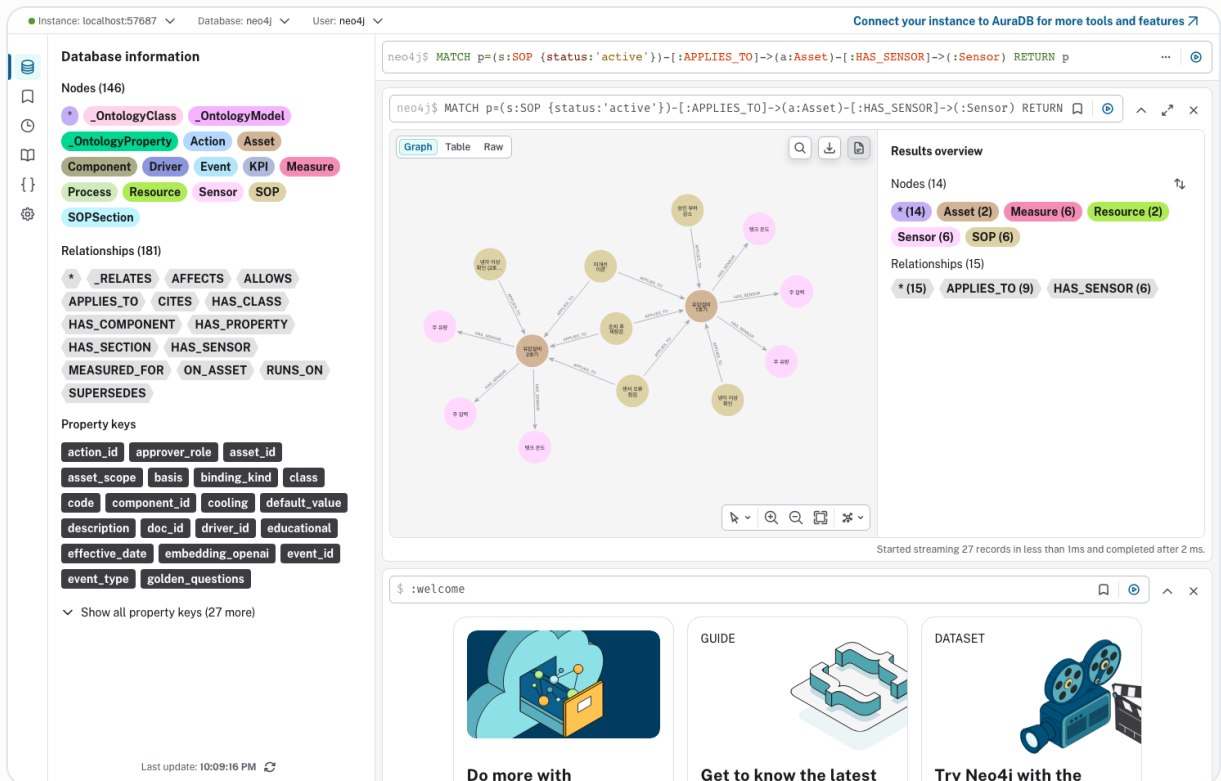
2. cypher-shell 에서 파라미터를 넘겨 실행한다.

```
docker compose exec -T neo4j cypher-shell -u neo4j -p hydops-lecture \
-P "{asset_id: 'HYD-02', event_type: 'COOLING_ANOMALY'}" \
"MATCH (sop:SOP)-[:APPLIES_TO]->(a:Asset {asset_id:\$asset_id}) WHERE
sop.status = 'active' AND sop.event_type = \$event_type OPTIONAL
MATCH (sop)-[:ALLWS]->(act:Action) RETURN sop.doc_id AS doc_id,
sop.version AS version, collect(act.action_id) AS actions ORDER BY
doc_id;"
```

```
doc_id, version, actions
"SOP-COOL-002", 1, ["FAN_BOOST"]
```

```
"SOP-ESC-001", 1, ["ESCALATE"]
"SOP-VER-001", 1, []
```

3. Neo4j Browser 에서 active SOP → 설비 → 센서 경로를 그림으로 본다.



Neo4j Browser 경로

`MATCH p=(s:SOP {status:'active'})-[:APPLIES_T0]->(a:Asset)-[:HAS_SENSOR]->(Sensor) RETURN p` 결과는 노드 14(Asset 2 · Sensor 6 · SOP 6), 관계 15(APPLIES_TO 9 · HAS_SENSOR 6)다. active SOP 가 없는 HYD-03 은 경로에 나오지 않고, 폐기 v1 의 APPLIES_TO 1개가 빠져 9개다.

4. 시작본 질의 세 개의 결과를 본다. `st · sol` 은 1회차와 같은 방식으로 불러온다.

```
import importlib.util
def load(path, name):
    spec = importlib.util.spec_from_file_location(name, path)
    mod = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(mod); return mod
st = load("../labs/practical/S03/context_queries.py", "st") # 시작
    본 (내가 고치는 파일)
sol = load("../labs/practical/S03/solution/context_queries.py", "sol") # 정답
    본
```

```
print("starter sensors HYD-02:", [r["sensor_id"] for r in
    st.asset_sensors("HYD-02")])
print("starter HYD-02 cooling:", [(r["doc_id"], r["version"],
    [a["action_id"] for a in r["allowed_actions"]]) for r in
    st.applicable_sops("HYD-02", "COOLING_ANOMALY")])
print("starter ctx HYD-03:", st.asset_context("HYD-03"))
```

```
starter sensors HYD-02: ['HYD-01.FS1', 'HYD-02.FS1', 'HYD-03.FS1', 'HYD-
01.PS1', 'HYD-02.PS1', 'HYD-03.PS1', 'HYD-01.TS1', 'HYD-02.TS1', 'HYD-
03.TS1']
starter HYD-02 cooling: [('SOP-COOL-001', 1, ['REDUCE_LOAD']), ('SOP-COOL-
001', 2, ['REDUCE_LOAD']), ('SOP-COOL-002', 1, ['FAN_BOOST']), ('SOP-ESC-
001', 1, ['ESCALATE']), ('SOP-LOAD-001', 1, ['REDUCE_LOAD']), ('SOP-VER-
001', 1, [])]
starter ctx HYD-03: {'asset_id': 'HYD-03', 'found': False}
```

HYD-02 의 냉각 이상 SOP 에 HYD-01 전용 SOP-COOL-001(폐기 v1 포함)과 SOP-LOAD-001 이 섞이고, 허용 조치에 REDUCE_LOAD 가 들어온다. 존재하는 설비 HYD-03 은 "설비 없음"이 된다.

5. 세 질의를 고친다.

```
// lecture/labs/practical/S03/solution/context_queries.py (정답본 발췌)
// 질의 1
MATCH (a:Asset {asset_id:$asset_id})-[:HAS_SENSOR]->(s:Sensor)
// 질의 2
MATCH (sop:SOP)-[:APPLIES_TO]->(a:Asset {asset_id:$asset_id})
WHERE sop.status = 'active' AND sop.event_type = $event_type
// 질의 3
MATCH (a:Asset {asset_id:$asset_id})
OPTIONAL MATCH (a)-[:HAS_COMPONENT]->(c:Component)
OPTIONAL MATCH (sop:SOP)-[:APPLIES_TO]->(a)
WHERE sop.status = 'active'
OPTIONAL MATCH (sop)-[:ALLOWS]->(act:Action)
RETURN a {.asset_id, .name, .cooling, .line} AS asset, c.component_id AS
component,
    collect(DISTINCT sop.doc_id + '@v' + toString(sop.version)) AS
active_sops,
    collect(DISTINCT act.action_id) AS allowed_actions
```

6. 정답본으로 네 설비 ID 를 조회한다.

```
for a in ("HYD-01", "HYD-02", "HYD-03", "HYD-99"):
    print(a, sol.asset_context(a))
```

```
print(" cool", [(s["doc_id"], s["version"], [x["action_id"] for x in
s["allowed_actions"]]) for s in sol.applicable_sops(a,
"COOLING_ANOMALY"]])
print(" sen", [(s["doc_id"], s["version"], [x["action_id"] for x in
s["allowed_actions"]]) for s in sol.applicable_sops(a,
"SENSOR_FAULT"]])
```

```
HYD-01 {'asset_id': 'HYD-01', 'found': True, 'asset': {'name': '유압설비 1호
기', 'cooling': '수랭식 냉각기', 'asset_id': 'HYD-01', 'line': 'L1'},
'component': 'HYD-01.COOLER', 'active_sops': ['SOP-COOL-001@v2', 'SOP-ESC-
001@v1', 'SOP-LOAD-001@v1', 'SOP-SEN-001@v1', 'SOP-VER-001@v1'],
'allowed_actions': ['ESCALATE', 'REDUCE_LOAD', 'SENSOR_CHECK']}
cool [('SOP-COOL-001', 2, ['REDUCE_LOAD']), ('SOP-ESC-001', 1,
['ESCALATE']), ('SOP-LOAD-001', 1, ['REDUCE_LOAD']), ('SOP-VER-001', 1, [])]
sen [('SOP-SEN-001', 1, ['SENSOR_CHECK'])]
HYD-02 {'asset_id': 'HYD-02', 'found': True, 'asset': {'name': '유압설비 2호
기', 'cooling': '공랭식 팬', 'asset_id': 'HYD-02', 'line': 'L2'}, 'component':
'HYD-02.COOLER', 'active_sops': ['SOP-COOL-002@v1', 'SOP-ESC-001@v1', 'SOP-
SEN-001@v1', 'SOP-VER-001@v1'], 'allowed_actions': ['ESCALATE', 'FAN_BOOST',
'SENSOR_CHECK']}
cool [('SOP-COOL-002', 1, ['FAN_BOOST']), ('SOP-ESC-001', 1,
['ESCALATE']), ('SOP-VER-001', 1, [])]
sen [('SOP-SEN-001', 1, ['SENSOR_CHECK'])]
HYD-03 {'asset_id': 'HYD-03', 'found': True, 'asset': {'name': '유압설비 3호기
(신규)', 'cooling': '수랭식 냉각기', 'asset_id': 'HYD-03', 'line': 'L3'},
'component': 'HYD-03.COOLER', 'active_sops': [], 'allowed_actions': []}
cool []
sen []
HYD-99 {'asset_id': 'HYD-99', 'found': False}
cool []
sen []
```

HYD-03 은 `found: True` 이면서 SOP 가 비어 있다 — 5회차의 "근거 없음 보류 (HOLD_NO_EVIDENCE)" 입력이다. HYD-99 는 설비 자체가 없다. 두 결과는 다르게 처리되어야 한다.

확인 포인트

- HYD-02 의 냉각 이상 SOP 는 SOP-COOL-002 v1 · SOP-ESC-001 v1 · SOP-VER-001 v1 뿐이고, 허용 조치에 REDUCE_LOAD 가 없다.
- HYD-01 결과에 SOP-COOL-001 v1 이 없다.
- 세 질의 모두 설비 ID 를 문자열로 박지 않고 `$asset_id` 파라미터로 받는다.

흔한 실수

- `WHERE a.asset_id = ' + asset_id + '` 처럼 문자열을 이어 질의를 만든다. 입력값이 질의 구조를 바꿀 수 있고, 질의 계획도 재사용되지 않는다.
- status 조건을 `WITH a, sop WHERE sop.status = 'active'` 처럼 `WITH` 뒤로 옮긴다. `OPTIONAL MATCH` 에 붙은 `WHERE` 는 매칭 조건이라 SOP 가 없으면 `sop` 을 null 로 남기지만, `WITH` 뒤의 `WHERE` 는 행 필터라 null 행을 지운다. HYD-03 으로 실행하면 `OPTIONAL MATCH ... WHERE` 는 `[{'id': 'HYD-03', 'sops': []}]` , `WITH ... WHERE` 는 `[]` 가 나온다.

6. 활동 4 — asset_id로 그래프와 시계열 결과를 결합 · 50분

목표

`join_recent(asset_id, run_id)` 를 고쳐 그래프의 센서 목록·SOP 와 PostgreSQL 최근 60 초 요약을 한 결과로 만든다.

따라 하기

1. 결합할 두 쪽의 키를 확인한다.

쪽	설비 키	센서 키	예
Neo4j <code>Sensor</code>	<code>(Asset {asset_id})</code> 경로	<code>code (sensor_id</code> 는 전역 ID)	<code>HYD-02.TS1</code> / <code>TS1</code>
PostgreSQL <code>observation</code>	<code>asset_id</code> 열	<code>sensor_id</code> 열	<code>TS1</code>

2. 시작본은 그래프 `sensor_id` 로 관측 요약을 찾고, `run_id` 를 넘기지 않는다.

```
# 시작본 발췌
rows = store.recent_window(asset_id, seconds, until=until)
summary = recent_summary(rows)
for s in asset_sensors(asset_id):
    obs = summary.get(s["sensor_id"])
```

3. 실습 데이터를 준비한다. UCI 사이클 100 을 HYD-02 의 `LAB-P03-demo` run 으로 재생 시각 2026-09-03 에 넣는다(수업 서버의 현재 시각보다 이르게).

```
store.create_run("uci_replay", "lab-s03", note="S03 결합 실습", run_id="LAB-
P03-demo")
store.insert_observations("LAB-P03-demo",
    SensorQualityChecker().check_many(uci.cycle_observations(ds, 100,
    "HYD-02", T0)))
```

4. 시작본 결합 결과는 센서 9개가 모두 비어 있다.

```
for s in st.join_recent("HYD-02", "LAB-P03-demo", 60)["sensors"]:
    print("starter", s["sensor_id"], s["samples"], s["unit_obs"])
```

```
starter HYD-01.FS1 0 None
starter HYD-02.FS1 0 None
starter HYD-03.FS1 0 None
starter HYD-01.PS1 0 None
starter HYD-02.PS1 0 None
starter HYD-03.PS1 0 None
starter HYD-01.TS1 0 None
starter HYD-02.TS1 0 None
starter HYD-03.TS1 0 None
```

(출력: 센서, 결합된 samples, 관측 단위.) 질의 1 이 고쳐지지 않아 센서가 9개이고, 결합 키가 달라 아무것도 붙지 않았다.

5. `run_id=run_id` 를 넘기고 `summary.get(s["code"])` 로 고친 정답본 결과다.

```
out = sol.join_recent("HYD-02", "LAB-P03-demo", 60)
print("context:", out["context"])
print("cooling sops:", [(s["doc_id"], s["version"]) for s in
    out["cooling_sops"]])
for s in out["sensors"]:
    print(s)
```

```
context: {'asset_id': 'HYD-02', 'found': True, 'asset': {'name': '유압설비 2호
기', 'cooling': '공랭식 팬', 'asset_id': 'HYD-02', 'line': 'L2'}, 'component':
'HYD-02.COOLER', 'active_sops': ['SOP-COOL-002@v1', 'SOP-ESC-001@v1', 'SOP-
SEN-001@v1', 'SOP-VER-001@v1'], 'allowed_actions': ['ESCALATE', 'FAN_BOOST',
'SENSOR_CHECK']}
cooling sops: [('SOP-COOL-002', 1), ('SOP-ESC-001', 1), ('SOP-VER-001', 1)]
{'sensor_id': 'HYD-02.FS1', 'code': 'FS1', 'quantity': 'flow', 'unit_graph':
'L/min', 'unit_obs': 'L/min', 'unit_match': True, 'samples': 60,
'valid_ratio': 1.0, 'valid_mean': 6.6, 'last_valid': 7.88, 'sensor_state':
None}
{'sensor_id': 'HYD-02.PS1', 'code': 'PS1', 'quantity': 'pressure',
```

```
'unit_graph': 'bar', 'unit_obs': 'bar', 'unit_match': True, 'samples': 60,
'valid_ratio': 1.0, 'valid_mean': 157.14, 'last_valid': 147.34,
'sensor_state': None}
{'sensor_id': 'HYD-02.TS1', 'code': 'TS1', 'quantity': 'temperature',
'unit_graph': '°C', 'unit_obs': '°C', 'unit_match': True, 'samples': 60,
'valid_ratio': 1.0, 'valid_mean': 53.26, 'last_valid': 53.4, 'sensor_state':
'VALID'}
```

그래프 단위와 관측 단위가 세 센서 모두 일치한다. 세 센서 모두 유효 비율 1.0 이다. FS1 의 9초째 0 → 8 L/min 펌프 기동은 정상 사이클 실측에서 정한 급등 기준(15.0 L/min) 안이므로 SPIKE 가 아니다(1회차 활동 3). `sensor_state` 는 `recent_summary` 가 TS1 에만 계산한다.

6. 같은 함수에 `run_id=None` 을 주면 무엇이 나오는지 비교한다.

```
o3 = sol.join_recent("HYD-02", None, 60)
print("solution run_id=None:", [(s["code"], s["samples"], s["valid_mean"])
    for s in o3["sensors"]])
```

```
solution run_id=None: [('FS1', 60, 9.0), ('PS1', 60, 149.76), ('TS1', 60,
45.0)]
```

HYD-02 의 가장 최근 관측은 수업 서버 시뮬레이터 run 이므로 유량 9.0 · 압력 149.76 · 온도 45.0 이 나왔다. 결합은 성공했지만 우리가 넣은 UCI 재생 데이터가 아니다. 결합 키에는 `asset_id` 와 함께 run 범위가 필요하다.

7. 완성 시스템의 Q2 답과 대조한다.

```
PYTHONPATH=., .venv/bin/python -c "
from hydops.golden import q2
print(q2('HYD-01')); print(q2('HYD-02')); print(q2('HYD-03'))
"
```

```
{'sop': [{'doc_id': 'SOP-COOL-001', 'version': 2, 'effective_date': '2026-09-01'}, {'doc_id': 'SOP-ESC-001', 'version': 1, 'effective_date': '2026-09-01'}, {'doc_id': 'SOP-LOAD-001', 'version': 1, 'effective_date': '2026-09-01'}, {'doc_id': 'SOP-VER-001', 'version': 1, 'effective_date': '2026-09-01'}], 'allowed_actions': ['ESCALATE', 'REDUCE_LOAD']}
{'sop': [{'doc_id': 'SOP-COOL-002', 'version': 1, 'effective_date': '2026-09-01'}, {'doc_id': 'SOP-ESC-001', 'version': 1, 'effective_date': '2026-09-01'}, {'doc_id': 'SOP-VER-001', 'version': 1, 'effective_date': '2026-09-01'}], 'allowed_actions': ['ESCALATE', 'FAN_BOOST']}
{'sop': [], 'allowed_actions': []}
```

8. 자기 행을 지운다(observation → run 순).

확인 포인트

- 결합 결과 한 덩어리에 설비 속성·적용 SOP·센서별 최근 통계가 함께 있다 — 5회차 `get_asset_context` 와 `get_recent_window` 도구가 나눠 돌려줄 정보다.
- `unit_match` 가 모두 True 다.

흔한 실수

- 그래프 결과와 시계열 결과를 센서 순서(리스트 인덱스)로 짝짓는다. 그래프는 `ORDER BY s.code` (FS1, PS1, TS1), 관측 요약은 dict 라 순서 보장이 없다. 키로 결합한다.
- 결합 결과의 온도를 보고 "HYD-02 는 53°C 이므로 냉각 이상"이라 판정한다. 이 값은 UCI 사이클 100(냉각기 3%)을 HYD-02 이름으로 재생한 교육용 데이터이고, 판정은 4회차 감지 규칙이 한다.

7. 실습 과제

- 폴더: `labs/practical/S03/` — `TASK.md` , `context_queries.py` (시작본), `solution/context_queries.py` , `test_check.py`
- 수정할 곳(`# TODO`(학생) 6곳): `Q_ASSET_SENSORS` (설비에서 출발), `Q_APPLICABLE_SOP` (설비 묶기 + active), `Q_ASSET_CONTEXT` (OPTIONAL MATCH + active), `join_recent` (run_id 전달, `code` 로 결합)
- 검증 방법: 질의에 `$asset_id` 파라미터 사용, HYD-02 센서 3개, HYD-02 냉각 SOP 에 SOP-COOL-001·SOP-LOAD-001·REDUCE_LOAD 없음과 FAN_BOOST 범위 1.0~1.5(기본 1.3), HYD-01 에 v1 없음, HYD-03 `found: True` ·SOP 없음, HYD-99 `found: False` , `LAB-S03-` run 으로 HYD-02 에 넣은 60초가 세 센서 모두 결합되고 단위 일치. 그래프는 읽기만 한다.
- 확인 명령:

```
cd lecture/system && PYTHONPATH=. .venv/bin/python -m pytest
../labs/practical/S03 -q
```

시작본 실행 결과(요약):

```
E AssertionError: assert ['HYD-01.FS1'...'-03.PS1', ...] == ['HYD-02.FS1'...'HYD-02.TS1']
E AssertionError: assert ('SOP-COOL-001' not in {'SOP-COOL-001', 'SOP-COOL-002', 'SOP-ESC-001', 'SOP-LOAD-001', 'SOP-VER-001'})
E AssertionError: HYD-03 은 존재하는 설비다 (SOP 가 없을 뿐)
5 failed, 1 passed in 0.31s
```

모두 고친 뒤: 6 passed . 세 회차를 함께 확인하려면 LAB_SOLUTION 없이 `pytest ../labs/practical/S01 ../labs/practical/S02 ../labs/practical/S03 -q` 가 17 passed 여야 한다.

8. 완료 확인 체크리스트

- ☐ 세 Golden Question 마다 그래프 경로·필터·PostgreSQL 테이블을 표로 정했다.
- ☐ `Asset:Resource` , `Sensor:Measure` 라벨과 전역 센서 ID(`HYD-01.TS1`)를 조회로 확인했다.
- ☐ HYD-02 의 COOLING_ANOMALY 적용 SOP 조회 결과에 SOP-COOL-001·SOP-LOAD-001 이 없다.
- ☐ HYD-01 결과에 폐기된 SOP-COOL-001 v1 이 없다.
- ☐ HYD-03 이 "설비 있음·SOP 없음"으로 조회된다.
- ☐ 설비 필터를 뺀 시작본 질의에서 섞인 SOP 목록을 출력으로 남겼다.
- ☐ 그래프 센서와 최근 60초 관측을 `asset_id · code · run_id` 로 결합해 세 센서 모두 samples 60·단위 일치를 확인했다.
- ☐ `pytest ../labs/practical/S03 -q` 가 6 passed 다.

9. 회고 (10분)

1. HYD-02 에 SOP-LOAD-001 이 섞였다면 5회차 에이전트는 어떤 조치를 제안하고, 7회차에 다루는 조치 검사 `action_allowed_by_sop` 는 그것을 막을 수 있을까?
`hydops/b8_action/executor.py` 가 허용 조치를 `graph.applicable_sops(asset_id, event_type)` 로 읽는다는 점을 함께 생각한다.
2. 초기 시드에서는 `AFFECTS` 관계의 출처가 HYD-01 전용 문서로 세 설비 모두에 적힌 적이 있다. 지금 시드의 `basis_by_asset` 표 대신, 출처가 그 설비에 적용되는 SOP 인지 **그래프 질의로 검사**한다면 어떤 Cypher 를 쓰겠는가?

3. "설비 없음(HYD-99)"과 "근거 없음(HYD-03)"을 같은 빈 결과로 돌려주면 운영 화면에서 어떤 오해가 생기는가?

개인 설명 과제 — 내가 설명할 질의 또는 실패 분기 하나: (a) `Q_APPLICABLE_SOP` 의 설비 필터 누락으로 HYD-02 에 섞인 SOP 목록, (b) status 필터 누락으로 나온 폐기 v1, (c) 필수 MATCH 때문에 HYD-03 이 `found: False` 가 된 실패 분기, (d) `run_id=None` 결합이 시뮬레이터 값을 가져온 실패 분기 중 하나를 골라 질의 문장과 출력으로 2분 안에 설명한다.

강사 노트

시간 배분

시각	내용
14:00~14:50	활동 1 질문-관계 표 (팀별 질문 분담 25분, 설비별 SOP 조회·문서 대조 25분)
14:50~15:40	활동 2 MERGE·라벨 읽기 (적재 문장 해설, 라벨·버전·AFFECTS 근거 조회)
15:40~16:10	휴식 30분
16:10~17:00	활동 3 파라미터 Cypher 수정 (시작본 출력 → Browser 경로 → 수정)
17:00~17:50	활동 4 결합, pytest
17:50~18:00	회고

사전 준비

- 수업 전날 강사가 한 번 적재한다: `store.init_schema()` (reset 없이), `graph.seed_graph()`, `search.index_sections()`. 수업 중 학생은 `seed_graph` · `reset_graph` 를 호출하지 않는다.
- `graph.graph_stats()` 로 Asset 3 · Component 3 · Sensor 9 · SOP 7 · SOPSection 42 · Action 4 · APPLIES_TO 10 · ALLOWS 6 · AFFECTS 6 · SUPERSEDES 1 을 확인한다. `AFFECTS` 의 HYD-02 출처가 `SOP-C00L-002@v1 §2` 인지도 본다.
- Neo4j Browser(<http://localhost:57474>)에 S07 과 같은 경로 질의를 즐겨찾기로 저장해 둔다.
- 플랫폼 bootstrap 은 오늘 필요 없다(Ontology Studio 발행은 6회차).

장애 대응

- Neo4j 가 응답하지 않으면 활동 1·2 는 F08 그림과 S07 스크린샷, `graph.py` 코드 읽기로 진행하고, 활동 3·4 와 테스트는 복구 후 진행한다. 이 경우 완료 확인은 다음 회차 시작 20분에 보충해 기록한다.
- 학생이 실수로 `CREATE` 문을 실행해 중복 노드가 생기면, 강사가 해당 라벨·키만 확인 후 정리한다(전체 `reset_graph` 는 다른 팀의 사건 참조도 지우므로 쓰지 않는다).

용어

용어	뜻
Asset / Resource	수업용 설비 클래스 / Ontologic 관점의 자원. Asset 노드는 두 라벨을 함께 가진다
Sensor / Measure	센서 클래스 / Ontologic 관점의 측정. <code>sensor_id</code> 는 설비가 붙은 전역 ID, <code>code</code> 는 관측 테이블의 센서 ID
APPLIES_TO	SOP → 설비. SOP 를 설비별로 가르는 관계
ALLOWS	SOP → 조치. 허용 조치와 값 범위(min·max·default)
SUPERSEDES	새 버전 SOP → 옛 버전 SOP
active / superseded	SOP 버전 상태. 질의·검색은 active 만 쓴다
MERGE	키가 같으면 기존 노드를 쓰고 없으면 만드는 적재. 같은 값으로 여러 번 실행해도 결과가 같다
파라미터 Cypher	<code>\$asset_id</code> 처럼 값을 파라미터로 넘기는 질의. 문자열 이어 붙이기를 쓰지 않는다
OPTIONAL MATCH	일치하는 관계가 없어도 앞의 행을 남기는 매칭. SOP 없는 설비를 "설비 없음"으로 만들지 않는다
Golden Question	시스템이 끝까지 답해야 하는 세 질문(유효·지속 이상 / 적용 SOP·조치 / 근거·승인·회복)